Instructor Notes — Claude Code Workshop

Facilitator cockpit. Keep this open throughout the day.

Suggested Durations

These are the instructor's discretion — adjust freely to the group's pace, energy, and prior experience. No clock times are prescribed here; durations are relative.

Block	Suggested Duration	Notes
B1 Doorway & Setup	45–60 min	Expect stragglers; budget for install surprises
B2 The Loop & The Goal	50–60 min	Core loop; first real output; keep energy high
— BREAK —	15 min	Natural pause after first deliverable
B3a Turning the Dials (model/effort)	40–50 min	Before lunch — interactive, fast feedback
— LUNCH —	~90 min	Creates the "dials before / meter after" seam
B3b Reading the Meter (token notebook)	30–40 min	After lunch — quieter; notebook walkthrough
B4 Giving Claude Hands (MCP + GitHub)	60–75 min	Most technically rich block; allow slack
— SHORT BREAK —	10 min	
B5 A Second Cockpit: Antigravity	40–50 min	Light; keep fallback ready (see below)
B6 Make It Run Itself + Wrap	30–40 min	`/loop`, `/schedule`, reflection; end on time

Total guided time (excluding breaks/lunch): approximately 5.5–6.5 hours. A standard full-day format with a 90-minute lunch fits comfortably.

The B3 lunch seam is intentional: participants spend the morning making things happen fast (model switching, effort dials), step away for lunch, and return to the slower, more analytical notebook section with fresh eyes.

Block 1 Setup Troubleshooting

Node / npm not found - Mac: recommend Homebrew (`brew install node`) or the nodejs.org installer. Remind participants to open a *new* terminal after install. - Windows: the nodejs.org LTS `.msi` installer is the safest path. After install, restart PowerShell or Windows Terminal (not just the window — close and reopen). If `npm` still not found, check that Node was added to PATH (the installer should do this; tick the box if prompted).

`claude` **not found after** `npm install -g` - Check that npm's global bin directory is on PATH. Mac/Linux: `npm config get prefix` → add `<prefix>/bin` to PATH. Windows: typically `%APPDATA%\npm` — confirm it is in the user PATH.

Login / auth loop - `claude` opens a browser tab for OAuth. If the browser does not open, copy the URL it prints and paste manually. Once authenticated, return to the terminal; the session should activate automatically. - If the account is on a free tier, Claude Code will warn about rate limits. Prompt the participant to upgrade or switch to API billing.

Windows-specific quirks - Use Windows Terminal or PowerShell, not cmd.exe. Git Bash also works but MCP npm commands occasionally behave differently; steer toward PowerShell for the MCP blocks. - Long npm paths can trigger Windows MAX_PATH issues. Enabling Developer Mode (Settings → System → For Developers) removes the 260-character path limit and solves most of these silently.

Fallback for anyone still stuck after 10 minutes - Pair them with a working neighbour (observer role is valuable: they watch the loop, ask questions, contribute prompts). - Keep the demo visible on the projector so they follow along until resolved.

Antigravity Weekly-Quota Caveat

Google Antigravity's free tier now operates on a **weekly quota** — there is no committed permanent free tier. Quota limits and terms can change.

Re-verify limits approximately one week before each delivery of this workshop. Check: how many API calls or compute units per week are available on the free tier, whether a credit card is required, and whether the MCP endpoint URL in `fallbacks/b5-antigravity-mcp-config.json` is still current.

During B5, budget hands-on steps for scarcity: if quota runs out mid-block, participants pivot to reading the fallback config and discussing what the block would have shown. The learning objective (understanding multi-MCP agent behaviour) survives without live execution.

Block 5b — NotebookLM (free bonus)

`exercises/b5b-notebooklm.md` is a hands-on card: participants upload the workshop's own PDF handouts (`pdf/`) into NotebookLM (free, any Google account — the same one Block 5 needs) and generate grounded answers, a Study Guide, and an Audio Overview. Two facilitator notes:

- **Share the handout PDFs** (the `pdf/` folder) before this block so everyone has sources to upload. A public URL or a Google Doc also works as a source.
- **Audio Overview takes a minute or two** and can queue. Pre-generate one on your own account before the session as a fallback, and keep talking through the chat while theirs render. Free-tier daily limits apply. NotebookLM features verified 2026-06-21.

Per-Participant Cost Budget

Recommended: Pro or Max subscription (flat-rate)

For a workshop, a **Claude Pro or Max subscription** is the right choice. The entire day's hands-on work is covered by the flat monthly fee — no per-token bill, no surprises, no mid-session "you've exceeded your limit" messages.

Approximate workshop token consumption per participant (rough order of magnitude, not a guarantee): - B2–B3 analysis and iteration: ~50k–150k tokens - B4 MCP + GitHub work: ~30k–80k tokens - B5–B6 loops and scheduling: ~20k–50k tokens - **Total: roughly 100k–300k tokens for the day**, well within Pro/Max limits.

Alternative: API billing

If participants are on API billing (usage-based), rough cost at Sonnet pricing (input $3/M, output $15/M) for the hands-on day: - At 200k input + 50k output tokens: ~$0.60 + $0.75 = **~$1.35 per person** - Using Opus (input

$5/M, output $25/M): **~$2.25 per person** These are informal estimates. Deep reasoning or heavy looping pushes higher.

Token notebook real-run cost

The `notebook/token-consumption-lab.ipynb` ships pre-executed with sample data. If you want to run it live against the API during B3b:

```
export ANTHROPIC_API_KEY=sk-ant-...
jupyter nbconvert --to notebook --execute --inplace notebook/token-consumption-lab.ipynb
```

This makes a dozen small API calls and costs **a few cents** (typically under $0.10). Perfectly safe to run live; just requires `ANTHROPIC_API_KEY` set in the environment.

Bonus — Claude vs DeepSeek provider comparison (notebook cells 6–9)

The notebook ends with an optional cost comparison: the same prompt sent to Claude **and** to DeepSeek V4 Flash, each priced with its own tariff — the "DeepSeek moment" slide (B3) made concrete in real numbers (~60× cheaper for this task). The table and chart render from sample data with **no key**. The live cell needs a **second key,** **`DEEPSEEK_API_KEY`** (free signup at platform.deepseek.com) alongside `ANTHROPIC_API_KEY`, and costs a fraction of a cent. No DeepSeek key? Just present the pre-executed sample numbers — they make the same point.

Bonus — Three tiers, one task (`notebook/llm-tiers-colab.ipynb`)

A separate, self-contained **Colab** notebook runs the *same* product-labelling task on three tiers and charts the cost of each:

- **Gemini** via `google.colab.ai` — free, no key, **Colab-only** (`from google.colab import ai`). Free models `google/gemini-2.5-flash[-lite]`, monthly limit; cost shown as $0.
- **DeepSeek** — cheap, needs `DEEPSEEK_API_KEY`; the tier that "runs without you".
- **Claude** — premium, needs `ANTHROPIC_API_KEY`.

The teaching point is the **free-vs-key boundary**: free Gemini is ideal while you sit and click *Run*; the moment AI must run unattended (a script, a schedule — Block 6), you need a key, and DeepSeek shows that costs a fraction of a cent. The notebook ships pre-executed with an illustrative cost chart, and each tier **skips gracefully** when unavailable. To run live, open it in Colab and (for the paid tiers) set the keys — deferred to the facilitator, like the other notebook. Verified 2026-06-21.

Fallback Cues

Keep the `fallbacks/` folder open in a separate window.

When to reach for it	Artifact
B2: Claude silently produces a script that does not print the December peak, or participant's model is unresponsive	`fallbacks/b2-sales-summary.py` — run it directly; compare output with `fallbacks/b2-expected-output.md`
B2 quiet-failure check	If Claude's script runs without errors but the answer looks wrong, run `python fallbacks/b2-sales-summary.py` and confirm "Best month: 2024-12"

When to reach for it	Artifact
B4: GitHub MCP add command unclear or auth fails	`fallbacks/b4-mcp-config.json` — show the `.mcp.json` structure directly; participants copy-paste the relevant block
B4: PR not created or hard to demo	`fallbacks/b4-pr-example.md` — a pre-written PR description that illustrates what a successful outcome looks like
B5: Antigravity quota exhausted or config unclear	`fallbacks/b5-antigravity-mcp-config.json` — walk through the config on the projector; the discussion about multi-MCP orchestration stands without live execution

---

The Verify-Everything Refrain

Running does not mean correct. Repeat this throughout the day.

Every time Claude produces output, the verification prompt is: > "Running without error is not the same as being right. How do we check?"

Specifically for the sales analysis: always confirm that **December 2024 (2024-12) is the peak month** in the output. This is the known answer from `data/orders.csv`. If a participant's Claude run produces a different peak, something went wrong — the prompt, the data path, or the script logic. Walk back through it together. Use `python _build/check_data.py` to confirm the data is intact.

Verified-On Date

All commands, pricing, and Antigravity limits in this workshop were verified on **2026-06-19**.

Before each delivery, re-verify: - Claude Code CLI commands (`/model`, `/effort`, `/usage`, `/loop`, `/schedule`, `claude mcp add`) against the current Claude Code docs. - Token pricing (input/output per 1M tokens for Opus, Sonnet, Haiku, and DeepSeek V4 Flash — used in the B3b provider-comparison demo). - Antigravity free-tier weekly quota and MCP endpoint URL. - Node.js MCP package names (`@modelcontextprotocol/server-filesystem`, GitHub MCP URL) for any version or naming changes. - `google.colab.ai` free-tier availability and model names (`google/gemini-2.5-flash[-lite]`) for the three-tier Colab notebook. - NotebookLM free-tier limits, source types, and the Audio Overview / Study Guide feature names for the B5b card.